This document provides an **exhaustive structural index and architectural specification** for the entire **FitMaster AI** codebase. It explains the exact purpose of every single directory, Python module, HTML template, CSS asset, configuration file, and database entity.

1. Folder Structure & Purpose Guide

Folder Path	Category	Detailed Purpose & Responsibility
`FitMaster/`	Project Root Config	Core Django project module containing global settings, WSGI/ASGI configurations, and master URL routing.
`users/`	Primary Django App	Main application handling user authentication, role management (Customer, Trainer, Admin), views, database models, signals, and unit tests.
`users/management/`	Management Commands	Custom Django CLI commands, including data seeders (seed_data.py) to populate default packages, products, and admin users.
`users/migrations/`	Database Migrations	Database schema migration files tracking changes to models over time.
`accounts/`	Secondary App	Auxiliary user account package modularized for legacy or isolated user operations.
`templates/`	HTML Templates	Contains 38+ HTML template files rendered by Django views, structured into base layouts, customer, trainer, admin, and error views.
`templates/includes/`	Partial Templates	Reusable partial HTML components such as admin_sidebar.html and trainer_sidebar.html.
`static/`	Development Static Assets	Raw static files including custom CSS stylesheets (theme.css, navbar.css, etc.) and original images/icons.
`staticfiles/`	Compiled Production Assets	Compiled and optimized static directory automatically generated by WhiteNoise via collectstatic for production delivery.
`logs/`	Application Logs	Stores application logs (fitmaster.log) and error logs (errors.log) generated by Django's rotating file handlers.
`venv/`	Virtual Environment	Isolated Python virtual environment containing project dependencies (Django, WhiteNoise, ReportLab, Waitress, Gunicorn, etc.).

2. Root & Infrastructure File Index

Filename	File Type	Functionality & Content Description
`manage.py`	Python CLI Script	Django's command-line entry point used for running server, migrations, tests, and management commands.
`requirements.txt`	Dependency Manifest	Lists all Python packages required for production: Django, Pillow, python-dotenv, whitenoise, dj-database-url, waitress, gunicorn, psycopg2-binary, reportlab.
`.env`	Local Env Config	Stores active environment secrets (SECRET_KEY, DEBUG, ALLOWED_HOSTS, CSRF_TRUSTED_ORIGINS).
`.env.example`	Env Template	Template environment file provided for production deployments and team setups.
`.gitignore`	Git Ignore Rules	Excludes sensitive .env credentials, db.sqlite3, media/, staticfiles/, logs/, and venv/.
`Dockerfile`	Docker Build File	Multi-stage Docker container definition optimized for deploying FitMaster AI in production.
`docker-compose.yml`	Docker Compose	Orchestrates FitMaster AI web service, PostgreSQL database container, persistent storage volumes, and health checks.
`.dockerignore`	Docker Ignore Rules	Excludes virtual environments, caches, and local databases from Docker build context.
`db.sqlite3`	SQLite Database	Local development SQLite database storing user records, memberships, plans, and store products.

`DEPLOYMENT.md`	Markdown Guide	Comprehensive production deployment manual covering Docker, Render, Railway, AWS, and DigitalOcean.
`USER_MANUAL.md`	Markdown Guide	Complete user manual for Customers, Trainers, and Administrators.

3. FitMaster Core Module (FitMaster/)

File Name	Description & Architecture Role
`FitMaster/settings.py`	Master configuration file containing **dotenv loader**, security flags (XSS, No-Sniff, Frame options), **WhiteNoise static asset engine**, **dj-database-url PostgreSQL/SQLite abstraction**, email backend, and **rotating file logger system**.
`FitMaster/urls.py`	Root URL dispatcher including users.urls, Django admin endpoint (/django-admin/), media asset URL handlers, and custom error handlers (handler404, handler500, handler403).
`FitMaster/wsgi.py`	WSGI interface entry point for web servers like Gunicorn and Waitress in production.
`FitMaster/asgi.py`	ASGI interface entry point for asynchronous servers or WebSocket channels.

4. Database Models Index (users/models.py)

Model Name	Key Fields	Business Logic & Relationships
`Profile`	user, role ('customer', 'trainer', 'admin')	Extends Django User model via OneToOneField to enforce Role-Based Access Control.
`TrainerDetail`	user, category ('gym', 'yoga', etc.)	Stores specialized training categories for trainer accounts.
`Package`	name, duration_months, price, is_active	Defines gym membership subscription tiers (3 Month, 6 Month, 1 Year).
`Membership`	user, package, start_date, end_date	Tracks customer subscription lifecycle, start/expiry dates, and active flags.
`Payment`	user, plan, amount, card, transaction_type	Logs financial transactions for memberships and supplement store purchases.
`Client`	trainer, client, goal, joined_on	Maps trainer-to-customer assignments and tracks primary fitness goals.
`WorkoutPlan`	trainer, client, title, description, duration	Custom workout routines assigned by trainers to specific clients.
`DietPlan`	trainer, client, breakfast, lunch, dinner	Customized daily nutrition and meal plans assigned to clients.
`Schedule`	trainer, client, session_date, session_time	Manages training appointments and workout sessions.
`Progress`	trainer, client, weight, height, bmi, notes	Trainer-logged fitness evaluation metrics and notes for clients.
`Measurement`	customer, weight, height, bmi, month	Customer self-logged body weight/height logs with automatic BMI calculation.
`Feedback`	customer, rating, message	Customer feedback ratings (1-5 stars) and reviews.
`Notice`	subject, message, recipient_role, email_sent	System-wide broadcast announcements sent via email.
`Product`	name, price, description, image	Supplements and merchandise items available in the E-Commerce store.

5. Complete HTML Template Index (templates/)

Template Name	Rendered View	UI Purpose & Components
`base.html`	Global Base Layout	Master HTML wrapper with Google Fonts, FontAwesome, top navigation bar, and footer.
`dashboard_base.html`	Dashboard Shell	Base template for user dashboards incorporating sidebars and profile header.
`home.html`	views.home	Public landing hero page showcasing gym highlights, statistics, and CTA buttons.
`signup.html`	views.signup	User registration page supporting Customer and Trainer role selections.

`login.html`	views.user_login	Secure login page with authentication error messages.
`dashboard.html`	customer_dashboard	Customer portal showing active package, trainer, recent workouts, and BMI.
`trainer_dashboard.html`	trainer_dashboard	Trainer workspace showing today's sessions, client count, and quick creation buttons.
`admin_dashboard.html`	admin_dashboard	Admin control center with total user counts, revenue totals, and transaction tables.
`membership.html`	views.membership	Subscription plans pricing cards (3 Month, 6 Month, 1 Year).
`payment.html`	views.payment	Card checkout interface for memberships and store products.
`payment_success.html`	views.payment_success	Payment confirmation page with links to dashboard and invoices.
`invoice.html`	views.invoice	Printable tax invoice view displaying itemized payment breakdown.
`store.html`	views.store	E-Commerce store catalog displaying products, prices, and Buy Now buttons.
`customer_plans.html`	customer_plans	Customer view for assigned workouts, diets, height/weight forms, and feedback.
`workout_plans.html`	workout_plans	Trainer interface for creating and assigning workout plans.
`diet_plans.html`	diet_plans	Trainer interface for assigning meal routines to clients.
`schedule.html`	schedule	Trainer interface for scheduling workout sessions with clients.
`progress.html`	progress	Trainer interface for logging client BMI and fitness notes.
`my_clients.html`	my_clients	Trainer table listing assigned clients and their fitness goals.
`add_client.html`	add_client	Trainer form to select unassigned customers and assign training goals.
`users_list.html`	users_list	Admin table displaying all registered users and edit controls.
`edit_user.html`	edit_user	Admin form to modify user details, roles, active status, and membership dates.
`admin_packages.html`	admin_packages	Admin management view for adding/editing membership packages.
`admin_products.html`	admin_products	Admin management view for store product inventory.
`notices.html`	notices	Admin view for broadcasting mass email announcements.
`404.html`	custom_404	Custom 404 Page Not Found error page.
`500.html`	custom_500	Custom 500 Internal Server Error page.
`403.html`	custom_403	Custom 403 Access Denied error page.

6. CSS Stylesheet & Asset Index (static/css/)

CSS Stylesheet	Styling Responsibility
`static/css/theme.css`	Global color palette, CSS variables, body typography, glassmorphism containers, and button styling.
`static/css/navbar.css`	Top navigation bar positioning, logo formatting, menu links, and login/signup buttons.
`static/css/home.css`	Landing page hero section, feature cards, trainer spotlights, and call-to-action sections.
`static/css/auth.css`	Form inputs, glassmorphism auth cards, role dropdowns, and message alerts.
`static/css/membership.css`	Membership subscription pricing cards and feature checklists.
`static/css/dashboard_base.css`	Sidebar navigation, metric stat widgets, profile header cards, and content layout.
`static/css/responsiv.css`	Media queries for tablet and mobile responsive screen adaptations.